ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO PROJECT 3

Đề tài: Thiết kế và xây dựng

Hệ thống phát hiện cháy sử dụng Computer Vision và IoT

Họ và tên	Nguyễn Khổng Duy Hoàng
MSSV	20225130
GVHD	Nguyễn Minh Quân
	Trịnh Văn Chiến

Hà Nội, tháng 1 năm 2026

Lời nói đầu

Đô thị hóa diễn ra càng nhanh, nguy cơ cháy nổ tại các khu dân cư và nhà xưởng đang ngày càng trở thành một vấn đề cấp bách, điển hình như vụ cháy ở chung cư Khương Hạ vài năm về trước, đã gây thiệt hại nghiêm trọng về người và tài sản. Vậy nên, vấn đề đặt ra là mọi nhà, mọi chung cư đều cần một hệ thống báo cháy cấp tốc, với độ trễ cho phép bằng giây. Tuy nhiên, trong thực tế hiện nay, dường như chưa có một giải pháp nào đủ hiệu quả, trong khi các biện pháp truyền thống đều gặp vấn đề như kết nối, hoặc tính đúng sai, hoặc xử lý khi có sự cố ngay tại điểm cảnh báo.

Bên cạnh đó, sự phát triển khoa học kỹ thuật hiện nay đã mở ra những hướng đi mới cho vấn đề này. Việc kết hợp giữa khả năng nhận diện thông minh của trí tuệ nhân tạo (AI) và khả năng kết nối IoT (Internet of Things) chính là một hướng tiếp cận mới cho vấn đề này, giá thành sẽ giảm, tạo điều kiện tiếp cận cho nhiều nhà hơn.

Xuất phát từ thực tiễn đó, trong khuôn khổ học phần Project 3, em đã lựa chọn và thực hiện đề tài: “: Thiết kế và xây dựng Hệ thống phát hiện cháy sử dụng Computer Vision và IoT”. Đồ án này là bước tiến quan trọng trong việc tích hợp hệ thống, tối ưu hóa giao thức truyền thông MQTT và xử lý bài toán thời gian thực trên thiết bị phần cứng.

Để hoàn thành đồ án này, bên cạnh sự nỗ lực của bản thân, em đã nhận được sự giúp đỡ, chỉ bảo tận tình từ các thầy cô và các bạn. Em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy Nguyễn Minh Quân và thầy Trịnh Văn Chiến. Các thầy đã luôn nhiệt tình hướng dẫn, định hướng giải pháp kỹ thuật và đưa ra những lời khuyên giúp em hoàn thiện đề tài được tốt nhất.

Mặc dù đã rất cố gắng hoàn thiện, nhưng do giới hạn về thời gian và kinh nghiệm thực tiễn còn hạn chế, cùng với sự hạn chế của phần cứng, đồ án khó tránh khỏi những thiếu sót, rất mong nhận được những ý kiến đóng góp, chỉ dẫn của quý các thầy để đề tài có thể hoàn thiện và phát triển tốt hơn nữa và có thể đi vào thực tế trong tương lai.

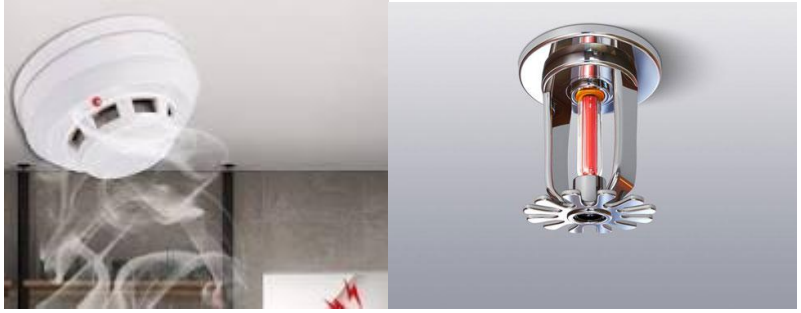
Em xin chân thành cảm ơn!

Chương 1: TỔNG QUAN ĐỀ TÀI

1. Tính cấp thiết của đề tài

- Trong những năm gần đây, tốc độ gia tăng dân số ở tại các đô thị lớn ở Việt Nam tăng rất nhanh, kéo theo đó là sự phát triển của thị trường nhà ở. Tuy nhiên, số lượng hiện nay chưa đi cùng với chất lượng, tuy số lượng nhà ở nhiều nhưng có những vấn đề cố hữu. Bên cạnh nhà nhỏ, xuống cấp thì vấn đề chống cháy được xem như một vấn đề tối quan trọng khi mà vụ cháy ở chung cư ở Hà Nội mấy năm trước xảy ra. Từ thực tế đó, em đã đi tìm hiểu về hệ thống báo cháy chung cư hiện nay, là tại sao đã có hệ thống báo cháy mà chung cư vẫn cháy, và làm sao để áp dụng nhân rộng.
- Theo thống kê từ Cục Cảnh sát PCCC và CNCH, chỉ trong tháng 1/2026, toàn quốc đã xảy ra 258 vụ cháy, làm chết 11 người, làm bị thương 6 người. Vậy nên, vấn đề đặt ra luôn phải là bất cứ nơi nào có nhà trọ con người ở, thì ở đó phải có thiết bị, hệ thống cảnh báo cháy.

- Tuy nhiên hiện nay, các hệ thống cảnh báo cháy chủ yếu vẫn thụ động và chậm chạp. Ví dụ như dựa vào cảm biến nhiệt độ, hay là dựa vào cảm biến khói, … hoặc các hệ thống chữa cháy tự động như bên dưới:



- Tuy nhiên, khi áp dụng vào thực tế, hệ thống này lại bộc lộ những điểm yếu khó khắc phục:
    - Hạn chế về môi trường hoạt động và độ trễ: Các cảm biến khói hoạt động dựa trên nguyên lý rất thụ động, tức là phải đợi khói từ đám cháy bay đến đủ nồng độ thì mới kích hoạt cảm biến, điều này là không phù hợp với các môi trường không gian mở như bãi gửi xe, khách sạn, khi mà gió có thể thổi khói đi mà không chạm vào cảm biến, có thể gây ra cháy nghiêm trọng trước khi cảm biến nhận ra.
    - Nhược điểm về chữa cháy tự động: Các hệ thống chữa cháy này thường có bầu thủy tinh cảm nhiệt, khi nhiệt độ tăng cao làm giãn nở chất lỏng, từ đó phá bầu thủy tinh để xả nước. Nhược điểm lớn nhất là dùng một lần, một khi kích hoạt, đầu phun phải thay mới. Hơn nữa, rủi ro kích hoạt sai là rất lớn. Đã từng có một du khách vô tình làm vỡ bầu thủy tinh trong quá trình sử dụng căn hộ, hậu quả là nước tràn ra khách sạn và phải bồi thường 140 triệu!
    - Vấn đề cảnh báo giả: Một số hệ thống cảm biến nhiệt hiện đại được lắp đặt cục bộ, thường có chi phí rất cao do yêu cầu thiết bị lớn. Bên cạnh đó độ chính xác thì không ổn. Chỉ là logic đơn giản, vĩ dụ nếu nhiệt độ lớn hơn 57 độ C thì sẽ kích hoạt báo cháy, gây khó chịu cho người sử dụng. Dần dà người dùng có xu hướng tắt cảnh báo đó đi.

- Sự ưu việt của giải pháp giám sát chủ động bằng thị giác máy tính: Từ những phân tích trên, ta có thể thấy các hệ thống truyền thống đang hoạt động khá thụ động (chờ tín hiệu đến). Em xin đề xuất một phương pháp tiếp cận chủ động với công nghệ Computer Vision.
    - Tốc độ phát hiện: Ngay khi lửa xuất hiện, mô hình AI có thể nhận diện trong vài mili-giây, nhanh hơn nhiều so với tốc độ khuếch tán của khói.
    - Khả năng phân biệt: Cảm biến nhiệt chỉ đo được độ nóng, cảm biến khói chỉ đo được nồng độ, AI có thể học được các đặc trưng hình ảnh của ngọn lửa, giúp hệ thống phân biệt được ngọn lửa thật và các nguồn nhiệt khác, ví dụ như đèn, mặt trời hay máy móc tỏa nhiệt, giảm thiểu tối đa báo động giả.
    - Giám sát từ xa: Hệ thống tích hợp IoT cho phép gửi hình ảnh thực tế về trung tâm xử lý, người dùng có thể xác minh ngay đó có phải cháy không và đưa ra điều khiển hệ thống chữa cháy, tránh thiệt hại do xả nước nhầm.

2. Mục tiêu nghiên cứu

- Đề tài đặt ra mục tiêu cốt lõi là giải quyết bài toán phát hiện sớm bằng cách khắc phục những nhược điểm về độ trễ và tính thụ động của mô hình truyền thống. Trong đó có các thành phần cụ thể:
    - Xây dựng hệ thống cảnh báo End-to-End: Thiết kế và triển khai một quy trình khép kín từ khâu thu thập dữ liệu hình ảnh đầu vào, xử lý ảnh, truyền tải thông điệp đến kích hoạt cảnh báo. Vấn đề đặt ra ở đây là xử lý đồng bộ giữa phần mềm và phần cứng.
    - Ứng dụng và tối ưu hóa mô hình Deep Learning: Nghiên cứu, ứng dụng các mô hình mạng nơ-ron tích chập CNN (ở đây sẽ là YOLOv8) để nhận diện chính xác đối tượng "lửa" (fire) trong các điều kiện môi trường thực tế, mục tiêu là độ chính xác mAP đạt trên 50% và độ nhạy Recall đạt trên 70% để giảm thiểu tối đa tỷ lệ báo giả.
    - Vấn đề về truyền tin và thời gian thực: Khắc phục độ trễ của các giao thức truyền thống như HTTP bằng cách triển khai giao thức MQTT (Message Queuing Telemetry Transport). Mục tiêu là đảm bảo độ trễ từ lúc phát hiện lửa đến lúc sáng đèn nhỏ hơn 2 giây.
    - Tích hợp IoT: Lập trình ESP8266 hoạt động để phát tín hiệu cảnh báo tại chỗ, đồng thời là ở chỗ cứu hỏa. Bên cạnh đó, cần xử lý các gói tin bảo mật để điều khiển thiết bị ngoại vi chính xác.

3. Đối tượng và phạm vi nghiên cứu

Để đạt được các mục tiêu đề ra, đề tài của em sẽ tập trung vào ba lĩnh vực chính:

1. Thị giác máy tính (Computer Vision) với YOLOv8
   Nghiên cứu kiến trúc mạng của mô hình YOLOv8, đánh giá hiệu năng và phạm vi triển khai.
   Tập trung vào quy trình huấn luyện, chọn dataset và đánh giá hiệu năng mô hình.
2. Lập trình nhúng ESP8266
   Nghiên cứu chip ESP8266 với khả năng kết nối WiFi tích hợp.
   Tìm hiểu về tài nguyên, cơ chế ngắt và tối ưu hóa hiệu năng để lập trình thiết bị có tài nguyên hạn chế.
3. Giao thức truyền thông IoT (MQTT & Security)
   Nghiên cứu cơ chế PubSub của giao thức MQTT.
   Tìm hiểu về các mức chất lượng dịch vụ (QoS 0, 1, 2) và cơ chế bảo mật đường truyền sử dụng mã hóa TLS/SSL để bảo vệ dữ liệu.

Do giới hạn về tài nguyên tính toán và nguồn lực hiện có, đề tài em được giới hạn cụ thể như sau:

1. Mô hình AI: Sử dụng phiên bản YOLOv8s (Small). Đây là phiên bản cân bằng giữa tốc độ và độ chính xác, phù hợp để triển khai trên các thiết bị máy tính cá nhân.
2. Phần cứng xử lý (Edge Device): Hệ thống sử dụng Laptop cá nhân, chip Intel i5 đóng vai trò là edge gateway để chạy thuật toán xử lý ảnh. Vi điều

khiển ESP8266 chỉ đóng vai trò là thiết bị nhận lệnh (Subscriber) và điều khiển đèn led, không chạy mô hình AI.

3. Môi trường thử nghiệm: Hệ thống được thiết kế và kiểm thử trong môi trường trong nhà.
4. Hiệu năng hệ thống: Phạm vi đáp ứng thời gian thực được xác định với tốc độ xử lý video đầu vào đạt 30FPS (Frames Per Second) và độ trễ truyền tin điều khiển nằm trong khoảng 2 giây.

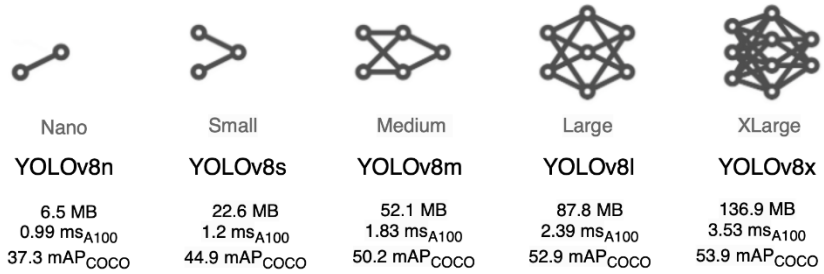
4. Bố cục đồ án

- Đồ án 3 của em được trình bày hệ thống qua 5 chương, trong đó có các nội dung cụ thể:
    - **Chương 1: Tổng quan đề tài** : Trình bày lí do chọn đề tài, phân tích tính khả thi, phân tích thực trạng và phạm vi nghiên cứu, triển khai của dự án.
    - **Chương 2: Cơ sở lý thuyết và Công nghệ** Cung cấp nền tảng kiến thức cần thiết để xây dựng hệ thống, em sẽ đi sâu phân tích kiến trúc mạng nơ-ron tích chập YOLOv8, nguyên lý hoạt động của giao thức MQTT, và kỹ thuật vi điều khiển ESP8266.
    - **Chương 3: Phân tích và Thiết kế hệ thống** Mô tả chi tiết kiến trúc tổng thể của hệ thống. Chương này bao gồm các sơ đồ khối, sơ đồ luồng dữ liệu, thiết kế thuật toán xử lý ảnh trên Python (bao gồm cơ chế tối ưu hóa luồng video và quản lý kết nối) và thiết kế firmware cho thiết bị nhúng.
    - **Chương 4: Xây dựng và Cài đặt chương trình** Trình bày quá trình thực hiện đồ án, từ khâu chuẩn bị dữ liệu, huấn luyện mô hình trên Google Colab, đến việc triển khai mã nguồn và tích hợp hệ thống thực tế.
    - **Chương 5: Thực nghiệm và Đánh giá kết quả** Phân tích các biểu đồ huấn luyện (Loss, mAP), đánh giá độ trễ thực tế và độ ổn định của hệ thống. Từ đó, rút ra kết luận về ưu nhược điểm và đề xuất hướng phát triển trong tương lai.

Chương 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1. Tổng quan về thị giác máy tính hiện đại và mô hình YOLOv8

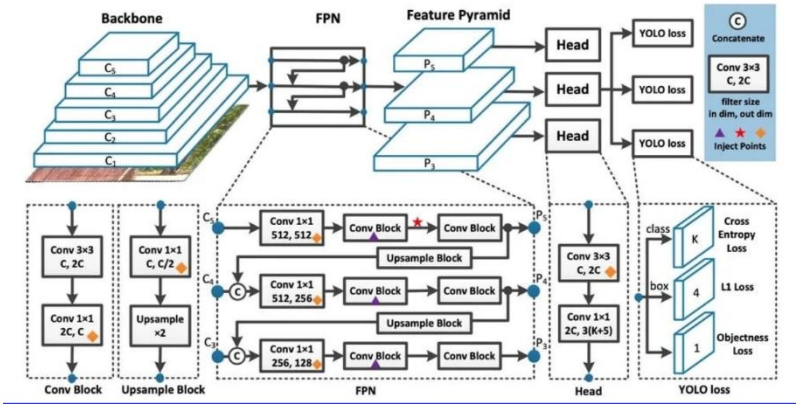
a. Sự phát triển của YOLO



- YOLO là họ các mô hình Object Detection theo thời gian thực nổi tiếng với tốc độ xử lý nhanh nhờ kiến trúc "một giai đoạn" (One-stage detector).
- So với với các mô hình R-CNN truyền thống phải quét qua ảnh nhiều lần, YOLO chỉ cần một lần lan truyền mạng (Forward pass) để dự đoán vị trí (Bounding Box) và phân loại đối tượng.
- Trong Project 3, hệ thống sử dụng phiên bản **YOLOv8** (ra mắt đầu năm 2023 bởi Ultralytics). Đây là phiên bản SOTA (State-of-the-Art) tại thời điểm thực hiện đồ án.

b. Kiến trúc mạng YOLOv8s

Dựa trên yêu cầu cân bằng giữa tốc độ và độ chính xác cho bài toán báo cháy, đồ án lựa chọn phiên bản **YOLOv8s (Small)** thay vì Nano (n) hay Medium (m).



- **Đặc điểm kiến trúc:**
  - **Backbone:** Sử dụng mạng CSPDarknet53 cải tiến với các khối C2f (Cross Stage Partial with 2 flow). Khối C2f giúp tăng cường khả năng trích xuất đặc trưng phong phú từ hình ảnh ngọn lửa (như màu sắc, biên dạng thay đổi liên tục) mà vẫn giữ được trọng lượng nhẹ hơn so với khối C3 cũ của YOLOv5.
  - **Head:** Sử dụng kiến trúc Decoupled Head (Đầu ra tách biệt). Việc tách riêng nhánh dự đoán phân loại (Classification) và nhánh dự đoán hộp giới hạn (Regression) giúp mô hình hội tụ nhanh hơn và chính xác hơn.

- **Anchor-free:** Loại bỏ các Anchor Box định sẵn, giúp mô hình linh hoạt hơn trong việc phát hiện các ngọn lửa có hình dạng bất định, không tuân theo tỷ lệ khung hình cố định.

- **Tối ưu hóa cho phần cứng Intel Core i5:**
  - Quá trình huấn luyện (Training) đòi hỏi khả năng tính toán song song lớn, nên được thực hiện trên đám mây sử dụng **GPU Tesla T4** của Google Colab.
  - Quá trình suy luận (Inference) thực tế diễn ra trên Laptop cá nhân sử dụng **CPU Intel Core i5**. Phiên bản YOLOv8s với khoảng 11.1 triệu tham số là sự lựa chọn tối ưu, đảm bảo tốc độ xử lý đạt ngưỡng thời gian thực khi kết hợp với thuật toán giới hạn tần suất (CHECK_INTERVAL=2.0) trong mã nguồn điều khiển.
  - YOLOv8s đạt tốc độ xử lý là 1.2ms cho mỗi ảnh khi chạy trên A100 của NVIDIA, đây là card xử lý mạnh mẽ với sức mạnh 312TFLOPS. Tuy nhiên, khi triển khai trên máy cá nhân, em chỉ dùng Intel i5, với sức mạnh chỉ khoảng 30GFLOPS, nên tính ra, tốc độ sẽ là 150ms-200ms khung hình một giây, nhưng điều này là hoàn toàn ổn bởi cứ 2s em mới cần xử lý một lần.

c. Hàm mất mát

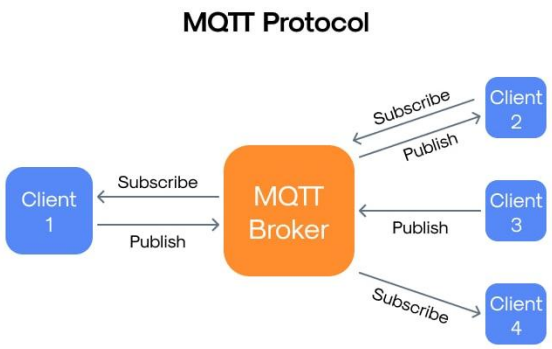
Để tối ưu hóa quá trình học, YOLOv8 sử dụng tổng hợp các hàm mất mát tiên tiến để đánh giá sai số giữa dự đoán của mô hình và nhãn thực tế. Điều này được thể hiện rõ qua quá trình hội tụ của các chỉ số box_loss, cls_loss và dfl_loss trong kết quả huấn luyện thực nghiệm.

Cụ thể, hàm mất mát tổng thể bao gồm 3 thành phần chính, và sẽ được tự động tính toán trong mô hình:

Hàm mất mát phân loại (Classification Loss - VFL)
Hàm mất mát hộp giới hạn (Box Regression Loss - CIoU)
Hàm mất mát phân phối (Distribution Focal Loss - DFL)

2. Hệ thống IoT và giao thức truyền thông

a. Giao thức MQTT



Mô hình Publish/Subscribe Để giải quyết bài toán giao tiếp thời gian thực giữa bộ xử lý AI (Laptop) và thiết bị nhúng (ESP8266), đồ án sử dụng giao thức MQTT thay vì HTTP truyền thống.

Publisher : Chương trình Python (run.py) sau khi phát hiện lửa sẽ gửi bản tin chứa trạng thái "ON" hoặc "OFF" tới một chủ đề (Topic) cụ thể, ví dụ: aegis/fire_alarm.

Broker : Sử dụng HiveMQ Cloud đóng vai trò là "bưu điện", chịu trách nhiệm nhận tin và chuyển tiếp ngay lập tức.

Subscriber : Vi điều khiển ESP8266 đăng ký nhận tin từ Topic aegis/fire_alarm và phản hồi ngay lập tức khi có dữ liệu mới.

b. Bảo mật

Quản lý định danh (Client ID): Trong môi trường mạng không ổn định, việc trùng lặp ID thiết bị dẫn đến hiện tượng "tranh chấp kết nối" . Hệ thống khắc phục bằng cách sử dụng thuật toán sinh ID ngẫu nhiên (uuid.uuid4()) cho mỗi phiên làm việc của Publisher, đảm bảo kết nối luôn duy trì ổn định.

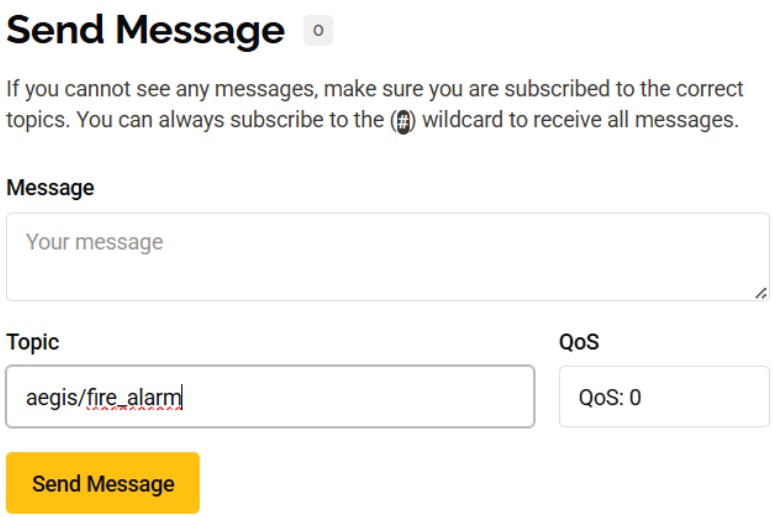
```
if "mqtt_client" not in st.session_state:
    # Tạo ID ngẫu nhiên để không bị trùng (Ví dụ: aegis-1234abc)
    random_id = f"aegis-{uuid.uuid4()}"

    client = mqtt.Client(
        client_id=random_id,
        protocol=mqtt.MQTTv311
    )
```

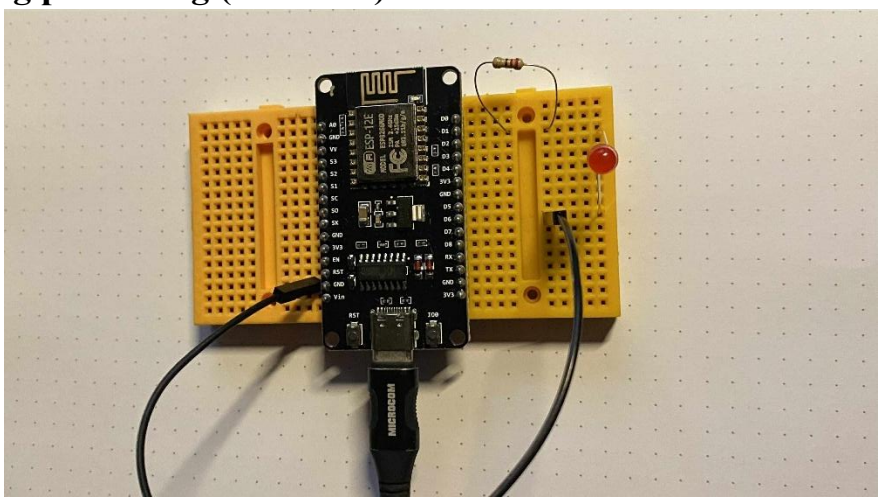
Mã hóa SSL/TLS: Để bảo vệ dữ liệu điều khiển khỏi các cuộc tấn công nghe lén , toàn bộ gói tin MQTT được mã hóa qua giao thức TLS tại cổng 8883.

```
// 2. Cấu hình HiveMQ lấy từ run.py
const char* mqtt_server = "40f1a9b7ef1f4ae9b2855dff153c4fb7.s1.eu.hiv
const int mqtt_port = 8883; // Cổng bảo mật
const char* mqtt_user = "Project3";
const char* mqtt_pass = "Project3";
const char* mqtt_topic = "aegis/fire_alarm";
```

Chiến lược QoS 0 (Quality of Service): Hệ thống sử dụng mức QoS 0 (At most once) cho chiều nhận của vi điều khiển. Lý do là ưu tiên tốc độ phản hồi thấp nhất cho báo động cháy. Việc mất một gói tin trong luồng dữ liệu liên tục (2 giây/lần) chấp nhận được hơn là việc hệ thống bị trễ do phải thực hiện bắt tay xác nhận (Handshake) phức tạp của QoS 1 hay 2. Trong chương trình thì khi mặc định khai báo PubSubClient thì mặc định QoS của mạch sẽ là 0.



3. Nền tảng phần cứng (ESP8266)



a. Giới thiêu Module ESP8266

Thay vì sử dụng các bo mạch cồng kềnh, đồ án sử dụng **Wemos D1 Mini**, một biến thể nhỏ gọn của dòng chip ESP8266EX.

- **Thông số kỹ thuật:** Vi xử lý 32-bit RISC @ 80MHz, tích hợp chuẩn Wifi 802.11 b/g/n.
- **Ưu điểm:** Kích thước siêu nhỏ, giá thành thấp (phù hợp triển khai đại trà), và hỗ trợ đầy đủ ngăn xếp giao thức TCP/IP cần thiết cho MQTT.

Hệ thống báo động vật lý được thiết kế tối giản để đảm bảo độ tin cậy:

- **Tín hiệu điều khiển:** Chân GPIO5 (tương ứng với chân **D1** trên bo mạch Wemos) được cấu hình là OUTPUT.
- **Mạch chỉ thị:** Một đèn LED báo động được kết nối với chân D1 thông qua một điện trở hạn dòng (220Ω - 1kΩ).
    - Trạng thái HIGH (Logic 1): Vi điều khiển xuất điện áp 3.3V, dòng điện đi qua điện trở làm sáng LED -> Báo động cháy.
    - Trạng thái LOW (Logic 0): Ngắt điện áp, LED tắt -> An toàn.

- **Lý do sử dụng điện trở:** Điện trở đóng vai trò bảo vệ, hạn chế dòng điện đi qua LED để tránh làm cháy LED hoặc hỏng chân GPIO của vi điều khiển khi hoạt động trong thời gian dài.

  Thách thức về tài nguyên bộ nhớ (Heap Memory) ESP8266 có bộ nhớ RAM khả dụng (Heap) rất hạn chế (khoảng 40KB). Việc thiết lập kết nối bảo mật SSL/TLS đòi hỏi bộ nhớ lớn để lưu trữ chứng chỉ. Trong phần mềm (sketch_jan29a.ino), giải pháp client.setInsecure() được áp dụng để bỏ qua bước xác thực chứng chỉ CA (Certificate Authority), giúp tiết kiệm tài nguyên bộ nhớ quý giá, ngăn chặn hiện tượng tràn bộ nhớ (Stack Overflow) mà vẫn giữ được lớp mã hóa dữ liệu đường truyền.

b. Thư viện WiFi và PubSubClient.

- Thư viện ESP8266WiFi (<ESP8266WiFi.h>):
  - Đây là thư viện cốt lõi cung cấp các lớp (Class) để điều khiển phần cứng Wifi của chip ESP8266/ESP-12E. Trong đồ án này, ta không sử dụng lớp WiFiClient thông thường (dành cho kết nối không bảo mật) mà sử dụng lớp WiFiClientSecure.
  - Vai trò: Thiết lập kết nối TCP/IP tới HiveMQ Cloud qua cổng 8883.
  - Tính năng quan trọng: Lớp WiFiClientSecure chịu trách nhiệm xử lý tầng giao vận bảo mật (Transport Layer Security - TLS). Đây chính là nơi thực thi hàm setInsecure() để bỏ qua xác thực chứng chỉ CA nhằm tối ưu hóa bộ nhớ Heap như đã phân tích.
- Thư viện PubSubClient (<PubSubClient.h>) :
  - Đây là thư viện phổ biến nhất để triển khai giao thức MQTT trên nền tảng Arduino/ESP8266.
  - Vai trò: Đóng gói và giải mã các gói tin MQTT (Connect, Publish, Subscribe) theo chuẩn OASIS.
  - Cơ chế hoạt động: Thư viện này hoạt động ở tầng ứng dụng (Application Layer). Nó không tự xử lý kết nối mạng mà phải dựa vào một "Client mạng" bên dưới.

4. Công cụ hỗ trợ

a. Roboflow

Đây là nền tảng lưu trữ dữ liệu tập trung. Tập dữ liệu em sẽ lấy từ trên này.
Ứng dụng thực tế: Đồ án sử dụng Roboflow để tiếp cận bộ dữ liệu "Fire Detection", thực hiện các thao tác tiền xử lý ảnh (Preprocessing) như: Resize về kích thước chuẩn đầu vào mạng YOLO (640x640 pixels) và Auto-orient để chuẩn hóa hướng ảnh.
Lợi ích: Giúp phân chia tập dữ liệu tự động theo tỷ lệ (Train/Val/Test) và xuất dữ liệu dưới định dạng tương thích trực tiếp với thư viện YOLOv8.

b. Google Colab

Vai trò: Cung cấp tài nguyên tính toán đám mây để huấn luyện mô hình Deep Learning.
Cấu hình phần cứng: Sử dụng GPU NVIDIA Tesla T4 (16GB VRAM) được cấp phát miễn phí trên Colab. Việc sử dụng GPU giúp giảm thời gian huấn luyện 20 epochs từ hàng chục giờ (nếu dùng CPU) xuống chỉ còn dưới 1 giờ.

c. HiveMQ Cloud

Vai trò: Đóng vai trò là MQTT Broker công cộng (Public Broker), trung chuyển tin nhắn giữa Laptop và ESP8266.
Đặc điểm kỹ thuật: Hỗ trợ giao thức MQTTS (Secure MQTT) qua cổng 8883. Đây là yếu tố then chốt giúp hệ thống vận hành được trên môi trường Internet thực tế mà không cần cấu hình NAT Port hay IP tĩnh phức tạp như khi tự dựng Server cục bộ.

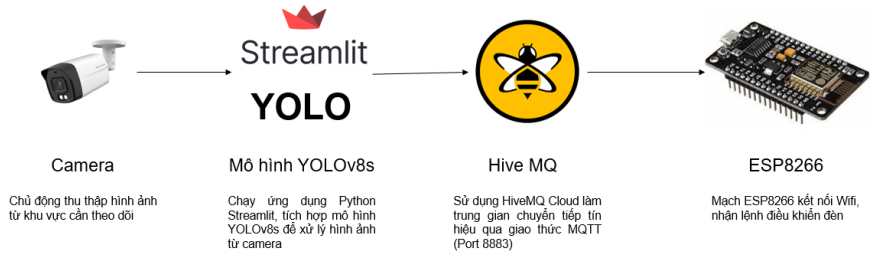
d. IDE

Visual Studio Code (VS Code): Sử dụng để phát triển module xử lý ảnh với ngôn ngữ Python. Các thư viện chính được tích hợp bao gồm: ultralytics (chạy mô hình AI), paho-mqtt (giao tiếp mạng) và streamlit (giao diện giám sát).

Arduino IDE: Sử dụng để biên dịch và nạp Firmware xuống vi điều khiển ESP8266. Thư viện PubSubClient và ESP8266WiFi được quản lý trực tiếp tại đây để xử lý logic điều khiển phần cứng.

Phần 3: PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

1. Sơ đồ kiến trúc (Architecture Diagram)



2. Thiết kế thuật toán tối ưu

Frame Skipping: Sử dụng tham số CHECK_INTERVAL = 2.0s để giảm tải cho CPU, tránh nghẽn cổ chai khi xử lý ảnh liên tục. Thực tế, với tốc độ khung hình là 30FPS, thì nếu bắt mô hình xử lý liên tục thì tín hiệu được đẩy lên broker với tốc độ chóng mặt, em cũng đã thử và terminal hiện ra liên tục dòng lệnh. Điều này thật sự không cần thiết, sau khi xem xét em để một biến CHECK_INTERVAL = 2.0, để mỗi 2s mới tiến hành xác minh xem có lửa không.

```
last_state = "SAFE"

# --- CẤU HÌNH THỜI GIAN ---
prev_time = 0
CHECK_INTERVAL = 2.0

# Biến lưu kết quả cũ
last_results = None
```

```
# --- LOGIC QUAN TRỌNG: CHỈ CHẠY AI SAU MỖI 2 GIÂY ---
if current_time - prev_time > CHECK_INTERVAL:
```

Bên cạnh đó, em đặt một biến last_state sẽ có tác dụng kiểm tra, nếu trạng thái cuối cùng khác trạng thái được trả về, lập tức sẽ so sánh và gửi tín hiệu tương ứng.

```
if current_state != last_state:
    msg = "ON" if is_fire else "OFF"
```

Session Management: Sử dụng uuid.uuid4() để sinh Client ID ngẫu nhiên, ngăn chặn xung đột kết nối (Connection Thrashing) khi reload ứng dụng. Điều này xảy ra khi mở nhiều cửa sổ ứng dụng. Nếu chưa đóng cửa sổ ứng dụng cũ (ở tab nào đó) thì khi mở tab mới, tab này sẽ đăng ký một cái id giống tab cũ (nếu không dùng uuid), khi ấy thì tab cũ sẽ mất quyền truy cập và sẽ cố gắng kick tab mới ra để lấy quyền kết nối, quá trình đấy lặp lại liên tục. Vậy nên em sinh ngẫu nhiên id cho chúng.

```
if "mqtt_client" not in st.session_state:
    # Tạo ID ngẫu nhiên để không bị trùng (Ví dụ: aegis-1234abc)
    random_id = f"aegis-{uuid.uuid4()}"

    client = mqtt.Client(
        client_id=random_id,
        protocol=mqtt.MQTTv311
    )
```

State Hysteresis: Cơ chế chỉ gửi tín hiệu MQTT khi trạng thái thay đổi (current != last), tránh spam tín hiệu xuống mạch.

```
if current_state != last_state:
    msg = "ON" if is_fire else "OFF"
```

3. Thiết kế Firmware

Resource Optimization: Sử dụng kỹ thuật setInsecure() để bỏ qua xác thực chứng chỉ CA, giúp tiết kiệm bộ nhớ Heap cho ESP8266 nhưng vẫn giữ lớp mã hóa dữ liệu SSL/TLS (Port 8883). Điều này là quan trọng, bởi bộ nhớ Heap của ESP8266 rất hạn chế, khoảng 40-50kB, nên nếu bắt xác thực bảo mật thì bộ nhớ sẽ không đủ làm việc đó. Vả lại, chúng ta cũng chỉ cấu hình broker với mật khẩu nên hoàn toàn tin tưởng được mà bỏ qua xác thực CA.

```
// --- QUAN TRỌNG: Bỏ qua kiểm tra chứng chỉ SSL ---
// Giúp kết nối nhanh và không bị lỗi bộ nhớ trên ESP8266
espClient.setInsecure();
```

QoS Strategy: Sử dụng mô hình hạ cấp QoS (Pub QoS 1 -> Sub QoS 0) để cân bằng giữa độ tin cậy và tốc độ phản hồi.

Phần 4: THỰC NGHIỆM & ĐÁNH GIÁ HỆ THỐNG

1. Chuẩn bị dữ liệu

Nguồn dữ liệu: Roboflow
Sô lượng: 1067 (Train: 872, Valid: 126, Test : 69)
Tiền xử lý: Resize 640x540, Auto-orient

```
#TẢI DATASET
rf = Roboflow(api_key="02lFbP1OG8B3wZJlk7Z9")
project = rf.workspace("hoangs-workspace").project("fire-detection-sejra-0vbl2")
version = project.version(1)
dataset = version.download("yolov8")
```

2. Quá trình huấn luyện

Môi trường: Google Colab (Tesla T4 GPU)
Tham số: 20 Epochs, batch size 16.

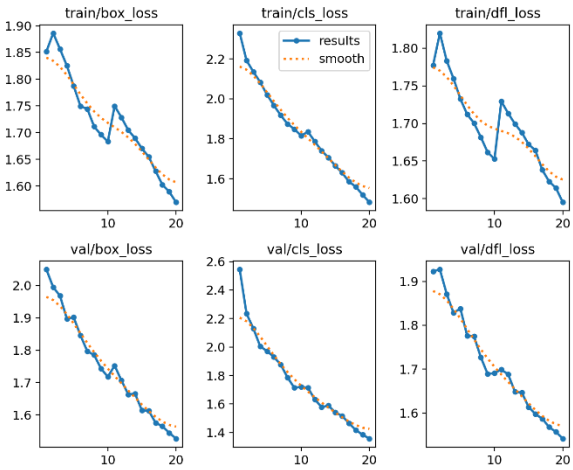
```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      20/20      6.47G       1.57      1.482      1.595         27        640: 100% ━━━━━━━━━━━━ 559/559 3.4it/s 2:43
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 280/280 4.1it/s 1:08
                   all       8939      24091      0.579      0.506       0.55      0.301

20 epochs completed in 1.323 hours.
Optimizer stripped from /content/runs/detect/train2/weights/last.pt, 22.5MB
Optimizer stripped from /content/runs/detect/train2/weights/best.pt, 22.5MB
```

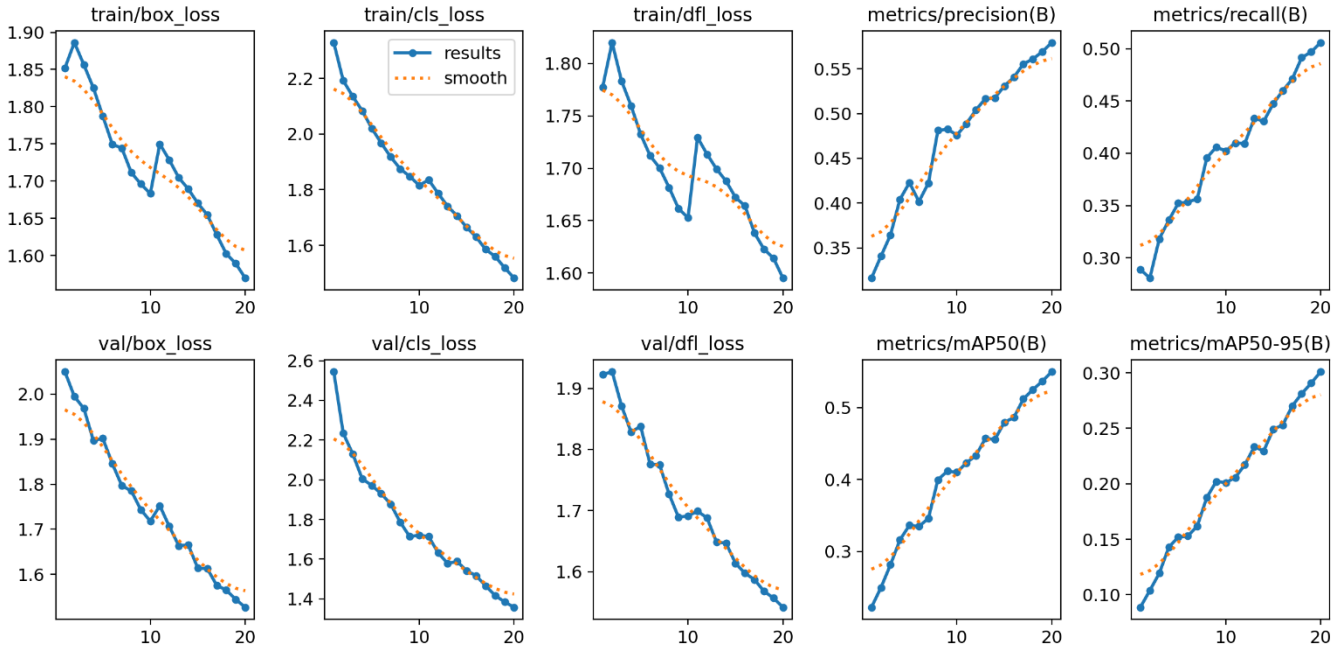
Cú nhảy Loss ở Epoch 10 là do cơ chế tắt Mosaic Augumentation, giúp model tinh chỉnh trên thực tế.
Trong hàm loss, ta cũng thấy ở giữa epochs 10 và 11 cũng có một khoảng loss tăng đột biến rồi giảm rất đều, chứng tỏ hiệu quả của phương pháp này.

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      10/20      6.47G      1.683      1.816      1.653         73        640: 100% ━━━━━━━━━━━━ 559/559 3.3it/s 2:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 280/280 4.1it/s 1:08
                   all       8939      24091      0.476      0.402       0.41      0.201
Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      11/20      6.47G       1.75      1.835      1.729         23        640: 100% ━━━━━━━━━━━━ 559/559 3.4it/s 2:45
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 280/280 4.0it/s 1:10
                   all       8939      24091      0.489       0.41      0.423      0.206
```



3. Kết quả mô hình



- Recall (Độ nhạy): ~70%. Nhấn mạnh chỉ số này quan trọng nhất với báo cháy (thà báo nhầm hơn bỏ sót).
- mAP@50: ~55%.

4. Kiểm thử hệ thống

Link demo: https://drive.google.com/drive/folders/11qOqgx-qljxtDTcY3ErdZt4yPRkzEBKf?usp=sharing

Phần 5: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Hạn chế:
Dữ liệu huấn luyện chỉ có nhãn là lửa, cần bổ sung thêm các lớp như 'smoke', hoặc các trường hợp nhiễu.
Số lượng ảnh, số lượng epochs huấn luyện còn ít để tối ưu cho khuôn khổ bài tập lớn.
Phục thuộc vào WiFi.

Hướng phát triển:
Tăng cường dữ liệu (có thể tăng thêm dữ liệu gây nhiễu như bóng đèn), số vòng huấn luyện epochs (lên 100-200).
Triển khai mô hình xuống biên để xử lý, giảm độ trễ thời gian cảnh báo.
Thêm module SIM để dự phòng nếu mất Wifi.
Chuyển publishers sang QoS 1 để chắc chắn gửi tín hiệu đến broker.

Tham khảo:

Documentation of Ultralytics YOLOv8

MQTT Essentials - HiveMQ Blog

Espressif ESP8266 Technical Reference).

Bài báo cáo Project3 của em đến đây là hết, em xin chân thành cảm ơn!